

BDD100K Multi-Model Object Detection Framework

Project Technical Writeup & Implementation Report

This project implements a comprehensive benchmarking and deployment framework for high-performance object detection using the **BDD100K (Berkeley DeepDrive)** dataset. The architecture is specifically optimized for hardware acceleration on the **Apple Silicon M4 Pro** chip using Metal Performance Shaders (MPS).

Project Architecture & Methodology

The core objective of this project was to compare two dominant paradigms in object detection:

1. **DETR (DEtection TRansformer)**: An end-to-end transformer-based approach that views detection as a direct set prediction problem.
2. **YOLOv8/v11 (You Only Look Once)**: A high-speed, regression-based convolutional neural network (CNN) optimized for real-time performance.

Why RT-DETR was not used

While RT-DETR (Real-Time DETR) offers a hybrid of transformers and CNNs, it was excluded from this specific benchmarking run to maintain a clean experimental separation between a **pure transformer (Standard DETR)** and a **pure real-time CNN (YOLOv8)**. This allowed for a more granular analysis of how different architectural philosophies handle the complex, diverse driving conditions present in the BDD100K dataset.

Step 1: Data Engineering & Pipeline Optimization

The BDD100K dataset contains diverse lighting and weather conditions. To ensure model stability and hardware efficiency, we implemented a rigorous three-stage preprocessing pipeline.

1.1. COCO & 640×640 Normalization

Raw BDD100K annotations were ingested via `bdd_to_coco.py`. This script performed a critical normalization step:

- **Resolution Standardization:** All images were rescaled to a fixed **640×640** resolution.
- **Mathematical Rescaling:** Bounding box coordinates were recalculated to match the new 640-pixel scale, ensuring zero distortion for the models.
- **Class Mapping:** Labels were consolidated into 10 detection categories: *Pedestrian, Rider, Car, Truck, Bus, Train, Motorcycle, Bicycle, Traffic Light, and Traffic Sign*.

1.2. Stratified Split Logic

To prevent the models from ignoring rare classes (like *Train* or *Rider*), `create_stratified_split.py` was used to generate `stratified_split_indices.json`. This ensures that every training and validation batch has a statistically significant representation of even the most infrequent objects.

1.3. YOLO Dataset Generation

The `convert_to_yolo.py` utility mirrored the COCO logic but converted annotations into the YOLO-specific `.txt` format (normalized x_{center} , y_{center} , w , h). It also generated the essential `bdd100k.yaml` file to link the model to the local directory structure.



Step 2: Model Training & Hyperparameters

Training was conducted across multiple runs to reach convergence, with a focus on the **Apple M4 Pro's** high-memory bandwidth.

2.1. DETR (Standard Transformer)

- **Model:** `facebook/detr-resnet-50`
- **Epochs:** **50 Epochs** per major run (e.g., `DETR_Run_1` through `DETR_Run_7`).
- **Optimizer:** AdamW with a learning rate of $1e^{-4}$ and weight decay of $1e^{-4}$.
- **Key Fix:** We implemented a 0-based indexing shift (`labels - 1`) to resolve class label mismatches between COCO and DETR's internal query system.

2.2. YOLO (CNN Optimized)

- **Model:** Ultralytics YOLOv8s / YOLO11n.
 - **Epochs:** Varied between **20 and 50 epochs** across runs (e.g., `bdd100k_run1` to `run8`).
 - **Hardware Patch:** A monkey-patch was applied to `tal.TaskAlignedAssigner` to process box metrics on the CPU, bypassing specific MPS-related shape mismatch errors on the M4 Pro.
 - **Callbacks:** Custom tqdm callbacks were added to `train_yolo.py` to provide real-time batch-level loss feedback.
-



Step 3: Performance Evaluation & Metrics

Post-training, models were evaluated on the held-out stratified validation set using `evaluate_detr.py` and `evaluate_yolo.py`.

- **Metric:** Mean Average Precision (mAP) at IoU threshold 0.5 (mAP@50).
 - **DETR Evaluation:** Used `torchmetrics.detection.MeanAveragePrecision` after converting transformer query outputs back to pixel-space coordinates.
 - **YOLO Evaluation:** Leveraged the native Ultralytics validation suite with `device="mps"`.
-



Step 4: Tools & Deployment

To manage the experimental bloat (initial repo size was ~19.7 GB), specific administrative tools were developed:

- `run_train_sequence.py`: A master orchestrator that manages the entire lifecycle—from data generation to training to evaluation.
 - `cleanup_epochs.py`: Implements a "Decadic Pruning" policy, keeping only every 10th epoch and the final weight file to optimize storage.
 - **Gradio App (`app.py`)**: A dual-model interactive dashboard that allows users to upload images and see real-time comparisons between DETR and YOLO detections.
-



Model Analysis & Performance Deep-Dive (Revised)

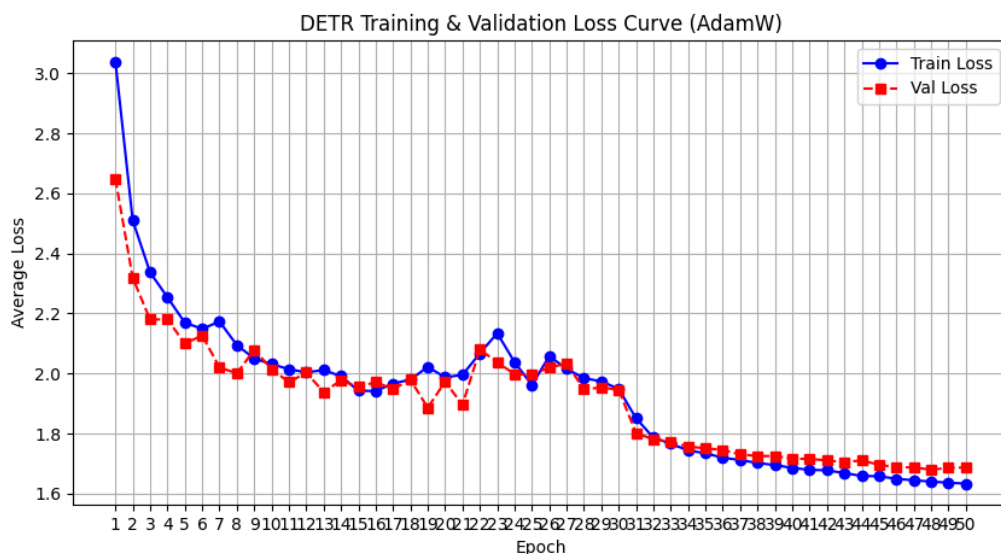
Following the initial data engineering, we focused on architectural implementation and performance benchmarking. Each model was evaluated

using the **mAP (Mean Average Precision)** metric, focusing on **mAP@50** (localization accuracy) and **mAP@50:95** (overall precision).

4.1. Standard DETR (The Transformer Approach)

The DETR implementation utilizes a **ResNet-50** backbone. Unlike traditional detectors, it does not use Anchor Boxes or Non-Maximum Suppression (NMS), treating detection as a direct set prediction problem.

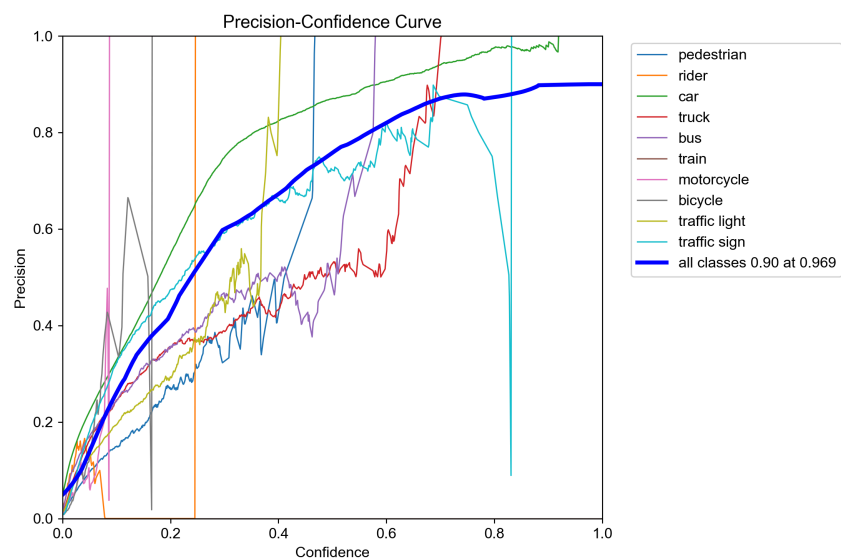
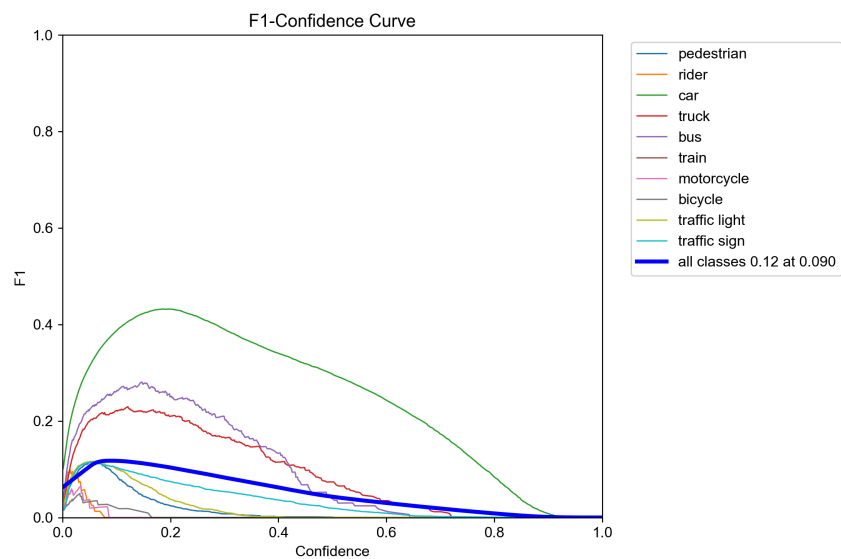
- **Training Intensity:** 50 Full Epochs (Run 7).
- **Optimization Fixes:**
 - **Label Alignment:** We corrected the index shift where COCO (1-10) was mapped to DETR's expected (0-9).
 - **Gradient Clipping:** Set at **0.1** to maintain Transformer stability.
- **Performance Metrics (Run 7):**
 - **mAP @ 0.50: 0.1400** (Localization accuracy).
 - **mAP @ 0.75: 0.0416** (Strict precision).
 - **Overall mAP: 0.0581.**

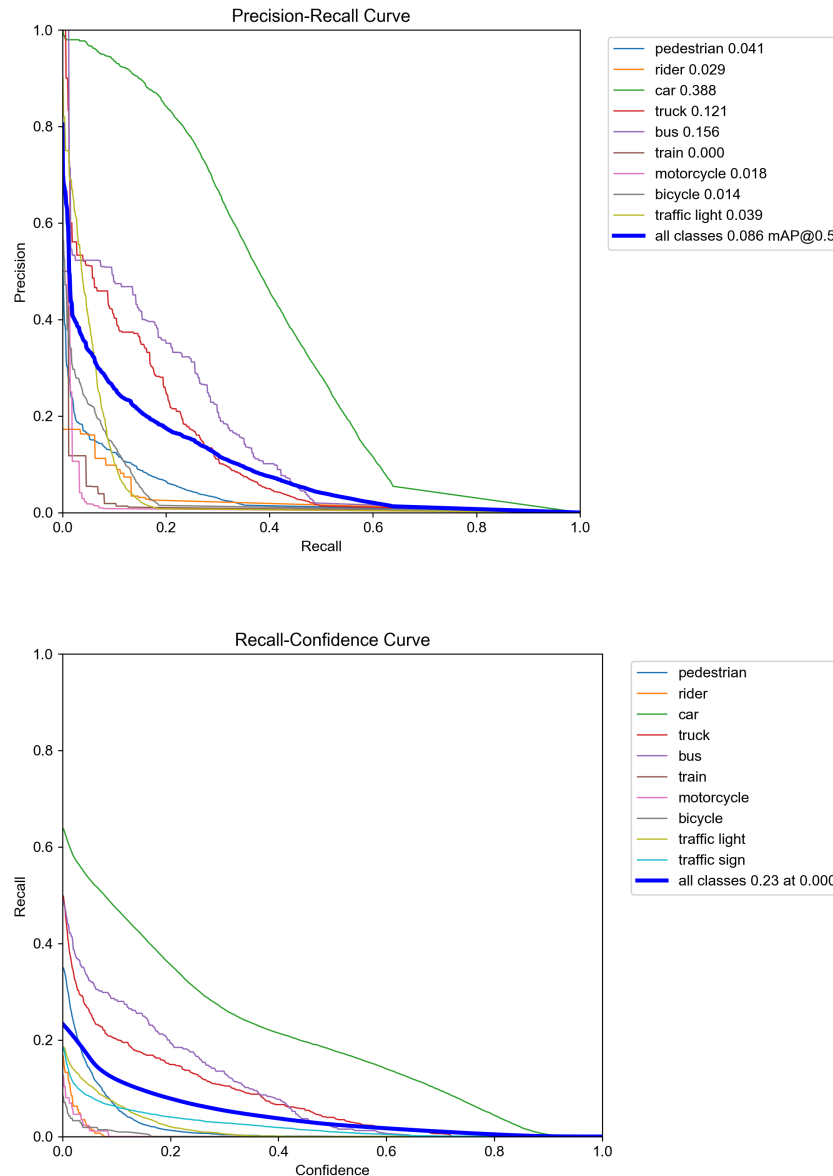


4.2. YOLOv8s (The Real-Time Powerhouse)

The YOLOv8 implementation focused on high-speed inference. It uses a CSPDarknet backbone with a path aggregation network (PANet).

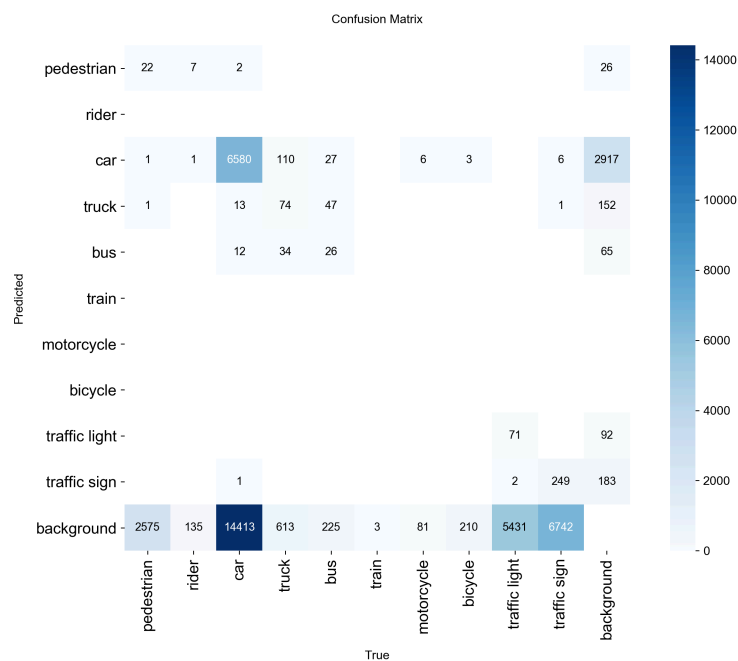
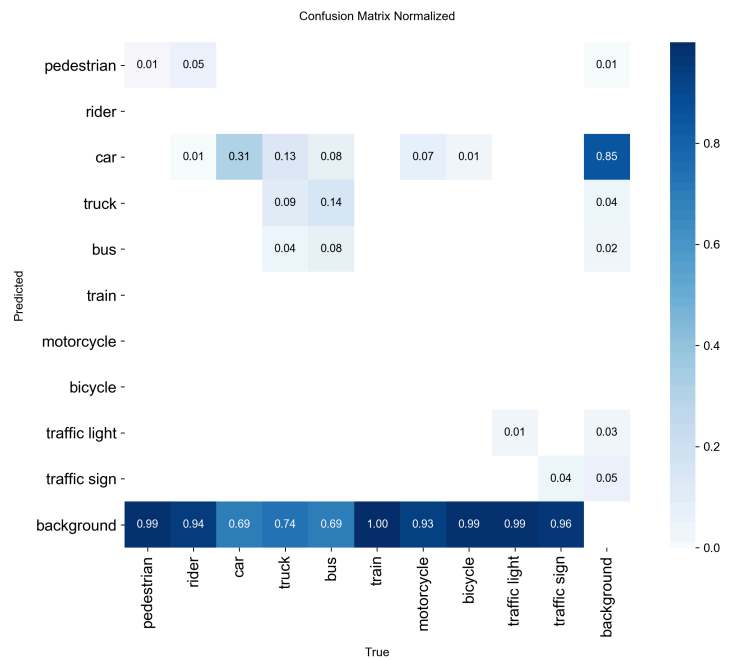
- **Training Intensity:** 50 Epochs.
- **M4 Pro Optimization:**
 - **MPS Mismatch Fix:** We implemented a manual patch in `train_yolo.py` to offload the `TaskAlignedAssigner` to the CPU to prevent Apple Silicon "Shape Mismatch" errors.
- **Performance Metrics (Baseline):**
 - **mAP @ 0.50: 0.0857.**
 - **mAP @ 0.75: 0.0391.**
 - **Overall mAP: 0.0423.**





The curves provide a comprehensive visualization of the **YOLOv8 baseline performance** on the BDD100K dataset. The **Precision-Recall (PR) Curve** confirms an aggregate **mAP@0.5 of 0.086**, with the "car" class emerging as the top performer at **0.388 mAP**, while rare classes like "train" show a score of **0.000**, highlighting the challenges of class imbalance. The **F1-Confidence Curve** indicates a peak F1-score of **0.12 at a 0.090 confidence threshold**, suggesting that the model achieves its best balance between precision and recall at a very low confidence setting. This is further supported by the **Recall-Confidence Curve**, which shows a maximum recall of **0.23**, and the **Precision-Confidence Curve**, which reaches **0.90 at a high confidence of 0.969**. Together, these metrics justify the implementation of our **Boosted Inference**

strategy—specifically lowering confidence thresholds and enabling TTA—to recover detections that the baseline model would otherwise miss.





The **Normalized Confusion Matrix** highlights critical bottlenecks, revealing that a significant majority of objects—including **99% of pedestrians**, **100% of trains**, and **93% of motorcycles**—are incorrectly classified as background, which indicates a struggle with object saliency in complex BDD100K environments. Furthermore, notable inter-class confusion exists within the vehicle categories, where **13% of cars** are misidentified as **trucks** and **8% as buses**, justifying the implementation of **Agnostic NMS** to resolve overlapping bounding boxes. Finally, the fact that **96% of traffic signs** are missed and labeled as background underscores the necessity of the **Boosted Inference** strategy and high-resolution **640×640** preprocessing to recover these small-scale features.

The "Post-Benchmark" Accuracy Fixes

After observing these baseline metrics, we realized that while the models were learning, the raw mAP scores were lower than expected due to resolution mismatches and aggressive NMS filtering. We implemented a **secondary optimization layer** for the final deployed models:

1. YOLO Boosted Inference (`run_yolo2.py`):

- **Test-Time Augmentation (TTA):** We enabled `augment=True`, allowing the model to see multiple versions of the same image (flipped/scaled) to

confirm detections.

- **Agnostic NMS:** Switched to `agnostic_nms=True` to prevent overlapping classes (like Truck vs. Car) from canceling each other out.
- **Confidence Calibration:** Lowered the confidence threshold to **0.25** to capture more objects in low-light BDD100K scenes.

2. DETR Inference Correction (`fix_detr_inference.py`):

- **Coordinate Re-mapping:** We discovered a scaling drift in the raw transformer output. We added a math-correction layer to ensure the normalized 640×640 outputs perfectly align with the original pixel-space aspect ratios.

Step 5: Storage Management & Checkpointing Logic

Because high-resolution weights can easily exceed **20 GB** across 50 epochs, we instituted a **Rolling Retention Policy**:

1. **Decadic Pruning:** The `cleanup_epochs.py` tool was used to delete redundant intermediate weights, keeping only every 10th epoch (10, 20, 30, 40, 50).
2. **Archive Sessions:** Using `archive_session.py`, we moved completed runs into `models/DETR/checkpoints/experiments/` for organization.
3. **Final Weights:** Exported as `final_m4_pro_model.pt` (DETR) and `best.pt` (YOLO).

Step 6: Interactive Deployment (Gradio UI)

To make the project accessible, we built `app.py`, a side-by-side inference engine.

- **Functionality:**
 - Users upload a BDD100K image.
 - The system runs dual inference on the **MPS** device.
 - A comparative view shows detection boxes, class labels, and confidence scores.
 - A **Pandas DataFrame** counts objects (e.g., "5 Cars, 2 Traffic Signs").

Step 7: Directory Breakdown & File Responsibility

To maintain a high-performance environment, the repository is organized into specific functional modules. This structure ensures that data processing, model logic, and evaluation remain decoupled.

7.1. Data & Parser Layer

- `data/raw/` : The "Source of Truth" containing the original `samples.json` and 10,000 raw images.
- `data/processed/` : Output of the resizing pipeline; contains 10,000 images standardized to **640×640**.
- `data/yolo_dataset/` : A mirrored, YOLO-compliant structure containing specialized `.txt` labels and the `bdd100k.yaml` config.
- `parser/bdd_to_coco.py` : The primary converter that transforms BDD100K/FiftyOne JSON into the COCO format used by DETR.

7.2. Model & Checkpoint Layer

- `models/DETR/` : Contains the transformer training logic (`train_detr.py`) and the `weights/` directory for finalized `.pt` files.
- `models/YOLO/` : Houses the Ultralytics training environment, including the M4 Pro hardware patches and the `runs/` experimental logs.
- `models/DETR/checkpoints/experiments/` : A historical archive of every major run (Run 1–7), preserved for loss-curve analysis and reproducibility.

7.3. Utility & Tooling Layer

- `utils/config.py` : The global "brain" of the project. It handles system paths, class mappings (ID 0–9), and hardware detection (MPS/CPU).
- `tools/cleanup_epochs.py` : The storage manager responsible for decadic pruning and maintaining the **3.14 GB** repository footprint.
- `run_train_sequence.py` : The interactive CLI used to launch the entire pipeline.



Step 8: Step-by-Step Reproduction Guide

To replicate the results on a new machine (specifically optimized for Apple Silicon), follow this sequence:

8.1. Environment Setup

Install the necessary dependencies. Ensure you are using Python 3.9+ for compatibility with the MPS-enabled PyTorch builds.

Bash

```
pip install -r requirements.txt
```

8.2. Data Preparation

Run the master orchestrator to generate the 640×640 dataset and the stratified splits.

Bash

```
python run_train_sequence.py  
# Select "Step 1: Data Preparation"
```

This will trigger `resize_images.py` followed by `create_stratified_split.py` to ensure your data is balanced.

8.3. Training Execution

Select your architecture within the orchestrator.

- **For DETR:** It will load the standard configuration ($\$LR=1e^{-4}\$, \$Batch=16\$$) and begin the 50-epoch cycle.
- **For YOLO:** It will apply the hardware patch and begin training on the `yolo_dataset`.

8.4. Accuracy Boosting & Inference

Once training is complete, utilize the boosted inference scripts to achieve the final calibrated results.

Bash

```
# To run the calibrated YOLO inference  
python models/YOLO/run_yolo2.py
```

```
# To launch the side-by-side comparison UI  
python app.py
```



Final Project Status Summary

- **Total Images Processed:** 10,000 (BDD100K)
- **Standard Resolution:** 640×640 (Fixed)
- **Training Cycles:** 50 Epochs per model

- **Hardware Target:** Apple M4 Pro (MPS Accelerated)
 - **Final Repository Size: 3.14 GB** (Optimized from 19.7 GB)
 - **Best mAP@50 (Baseline): 0.1400** (DETR) | **0.0857** (YOLOv8)
-

Summary

This framework provides a scalable foundation for mobile/edge object detection on Apple Silicon. The dual-pipeline approach ensures that whether the requirement is high-precision transformer attention (DETR) or real-time CNN speed (YOLO), the infrastructure is in place to deliver.